[딥러닝 경량화] 딥러닝 네트워크 양자화(quantization)는 왜 필요할까?

조재훈 2024.09.03 · 조회 7201

9 1

안녕하세요 현대자동차의 조재훈 책임연구원입니다.
이번 포스팅은 이전에 작성한 '[딥러닝 경량화] 실무에서 적용 중인 딥러닝 모델 경량화/최적화 기법은?'(https://developers.hyundaimotorgroup.com/blog/366) 에서 언급 드린 네트워크 경량화/최적화 기법 중 **양자화(Quantization) 기법**에 대해 이야기해 보는 시간을 가지려 합니다.

자율주행 차량을 탑승했다고 상상해 볼까요?

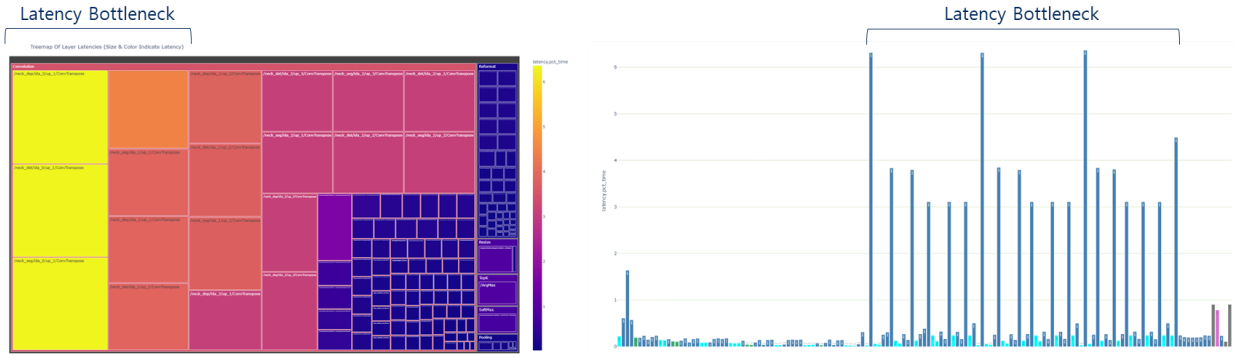
실시간으로 자율주행 차량이 차량 주변의 상황을 인지하고, 주변 상황 및 환경들을 판단한 후, 브레이크를 밟을지, 운행을 지속할지 제어하며 안전하게 목적지까지 안내해 줄 것 같은데요 저와 비슷한 상상을 하셨을까요? 앞서 말씀드린 것처럼 이때 자율주행 상황을 가능케 하는 많은 핵심 요소 중 한 가지는 '실시간 구동형'이라고 생각합니다. 그러나 에지 디바이스에는 메모리, 컴퓨팅 리소스, 전력이 제한되어 있습니다. 자율주행 차량도 마찬가지죠. 이는 딥 러닝 네트워크가 임베디드 배포에 최적화되어야 함을 의미합니다. 이때 활용하는 핵심 기술이 양자화 (Quantization)입니다.

양자화(Quantization) 란?

양자화 (Quantization)는 딥러닝 모델의 weight 값을 저장할 때 사용하는 비트(bits)를 줄여서 모델 크기를 줄이는 방법입니다. 일반적으로 딥러닝 모델을 학습할 때, 32개의 bit를 사용하는 32-bit floating point(FP32)로 모델을 저장합니다. Quantization은 weight 값을 저장할 때는 주로 Integer 8 (INT8)로 더 적은 bit를 사용하여 범위를 줄여주고, 표현 가능한 범위의 숫자로 변환한 뒤 해당 bit 수만큼의 메모리에 저장합니다. Quantization을 하면 더 적은 bit를 사용하기 때문에 모델의 메모리 사용량이 줄어서 추론(Inference) 시 동작시간을 단축할 수 있으며 하드웨어에서 딥러닝 모델을 구동하기 위해서 필수적인 과정입니다.
양자화는 일반적으로 **PTQ (Post Training Quantization)** 과 **QAT (Quantization Aware Training)** 2단계로 수행할 수 있는데요, 이번 포스팅에서는 Nvidia에서 제공하는 오픈 라이브러리를 활용하여 PyTorch quantization을 수행해 보도록 하겠습니다.

1) FP32 모델 프로파일링

기존 네트워크 (FP32) 구조를 파악을 하며, FP32에서 INT8로 변환이 될 때 Target 하드웨어사 지원이 되는지, convertin g issue가 있는지 확인합니다.
이때, layer 별 latency를 확인할 수 있으며, 대체 layer를 선정해서 모델 구조 수정이 가능합니다.
아래의 그림은 (Fig.1) 실제로 FP32 네트워크를 프로파일링 한 결과를 시각화한 이미지로, layer 별 latency를 확인할 수 있으며 bottleneck 부분의 경우 수식적으로는 동일하지만 latency 측면에서 이점이 있는 대체 layer로의 변경 혹은 m operator를 직접 설계할 수 있습니다.



(a) Treemap of Layer Latencies (b) Latency Budget Per Layer

Fig.1 FP32 모델 프로파일링

2) PTQ (post training quantization) 수행

기존 모델을 기반으로 training data (or validation data)를 활용하여 모델 및 data calibration을 수행합니다.
Data calibration 이란?

전체 Data의 distribution을 파악하여 데이터의 값의 범위가 int8 모델에 반영되는 과정입니다.
전체 데이터들의 대표성 (representative) 을 갖는 set으로 구성해야 합니다.

PTQ를 수행하기 위해서는 네트워크에 대한 이해가 필요합니다.
또한 하드웨어 제조사마다 제공하는 quantization 라이브러리의 특성이 다르기 때문에, 각 특성을 이해하여 양자화를 수행해야 합니다.
Nvidia 양자화의 경우 갖는 특성은 다음과 같습니다.

skip connection 및 concatenation 을 수행할 때 quantization이 적용되지 않음
이를 위해 manual 하게 quantization 수행 필요
예를 들어 resnet block과 같은 구조에 아래 참조와 같이 manual 하게 quantization 수행해야 함
참조: line 123 of https://github.com/NVIDIA/TensorRT/blob/96e23978cd6e4a8fe869696d3d8ec2b47120629b/tools/pytorch-quantization/examples/torchvision/models/classification/resnet.py

3) QAT (quantization aware training) 수행

학습된 PTQ 값을 baseline으로 QAT 수행

FP32 모델과 동일하게 학습 수행하는 데, 이때의 모델은 qdq (Quantize-Dequantize) layer 가 포함된 모델입니다. (아래의 그림 Fig.2 참조)
FP32 모델과 동일하게. pth 가 생성이 되며 model을 print 해보면, 추가적으로 quantized_ 와 같은 layer들이 생성되는 것을 확인해 볼 수 있습니다.
optimizer, scheduler 등의 hyper-parmeter에 대해 다양한 방식으로 실험 수행 필요합니다.

4) ONNX graph 결과 비교

아래의 Fig.2 그림은 FP32 모델과 quantization을 수행한 INT8 모델의 결과를 시각화해 보았습니다.
같은 영역을 시각화하였으며, 파란색 박스 부분에서qdq layer들이 적절히 삽입된 것을 확인하였습니다.
약 67%의 latency 저감 효과를 가져올 수 있었으며 양자화가 성공적으로 수행된 것을 확인하였습니다.

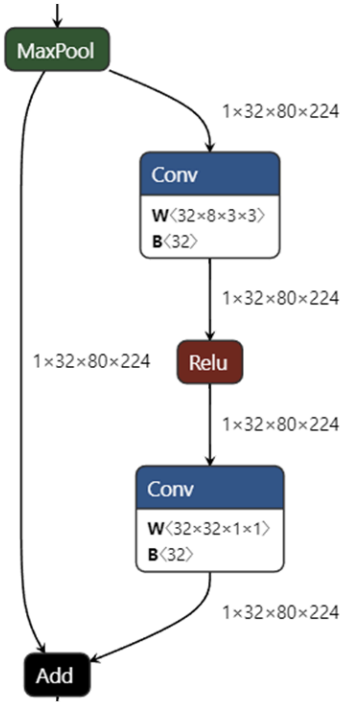
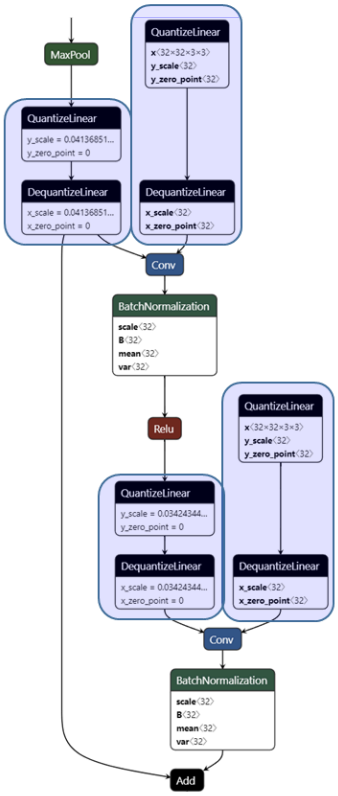
	FP32 모델	INT8 모델 (양자화 적용)
latency	55.257 ms	17.839 ms (-67%)
Onnx		

Fig.2 FP32 vs INT8 onnx 결과 비교

마무리

오늘은 nvidia에서 제공하는 toolkit을 활용하여, 양자화를 수행해 보았습니다. 조금 더 깊게 들어가면, PTQ 방식에서 1) 어떤 식으로 data calibration을 잘 수행할 있는지? 2) batch-norm folding (conv-relu-batch를 하나로 묶어주는 방식) 활용 방식, 3) 하드웨어 제조사마다 제공하는 PTQ 방식들 (cross layer equalization, adaround) 등 각 step마다 고려해 줘야 할 사항들이 많습니다.

최근에는 data calibration을 잘하기 위해 data distillaion 방식을 차용하여 특허출원을 했었는데요, 이처럼 양자화에서 고려해야 할 사항이 많기 때문에 항상 관심을 갖고 업무를 수행하려 하고 있습니다. 보다 더 자세한 내용이 궁금하시다면 댓글/연락 부탁드릴게요! 감사합니다.

> **자율주행 차량을 탑승했다고 상상해 볼까요?**

양자화(Quantization) 란?

> 1) FP32 모델 프로파일링
>
> 2) PTQ (post training quantization) 수행
>
> 3) QAT (quantization aware training) 수행
>
> 4) ONNX graph 결과 비교

마무리